線性特徵空間實驗

蔡秀吉 hctsai@linux.com

1 方法

合成資料依題目產生，三類各 100 筆，平均數為 $\mu_1 = (2,2,0)^\top$、$\mu_2 = (-2,2,0)^\top$、$\mu_3 = (0,-2,0)^\top$，三類共用 Σ_{signal}，後面再接上 q 維獨立的 $\mathcal{N}(0, 0.5)$ 雜訊（$q = 0$ 到 100）。不同的 q 共用同一批亂數，只差取了幾欄。訓練集與測試集依類別分層抽出，比較 50/50 與 20/80 兩種比例。每個設定重複 20 次，每次重新產生資料與分割，文中的數字是 20 次的平均。平均數、共變異數、標準化、PCA 與 LDA 都只用訓練集估計，再套用到測試集。

1-NN 以歐氏距離找最近的訓練點。訓練準確率讓訓練點和自己比對，所以一定是 100%，圖 2 另外畫出排除自己的 leave-one-out 準確率。第二個分類器（下文稱 MD）把樣本分給 Mahalanobis 距離最近的類別平均數：

$$d_c^2(x) = (x - \hat{\mu}_c)^\top \hat{\Sigma}_c^{-1} (x - \hat{\mu}_c),$$

其中 $\hat{\mu}_c$ 與 $\hat{\Sigma}_c$ 是第 c 類的樣本平均數與共變異數（除以該類的訓練筆數 n_c）。這是講義中的最近平均數分類器，每類用自己的共變異數。$\hat{\Sigma}_c$ 的秩最多是 $n_c - 1$，維度 $d \geq n_c$ 時一定奇異，這時改用擬反矩陣 $\hat{\Sigma}_c^+$。

PCA 在訓練集上分解 $\hat{\Sigma} = V \Lambda V^\top$，保留特徵值最大的 k 個方向，並加入 ridge：

$$z = (\Lambda_k + \gamma I)^{-1/2} V_k^\top (x - \hat{\mu}), \qquad \gamma \in \{0,\ 0.01,\ 0.1,\ 1,\ 10\}.$$

$\gamma = 0$ 是一般的 PCA 白化，只在保留的特徵值都大於 0 時有定義。LDA 使用講義的散布矩陣 S_w 與 S_b，解 $S_b w = \rho (S_w + \gamma I) w$ 後取前 $C - 1 = 2$ 個方向。$\gamma > 0$ 時即 regularized LDA，下文寫作 rLDA。S_w 奇異而 $\gamma = 0$ 時，只在 S_w 的值域內求解。估計與分類都以 NumPy 寫成，scikit-learn 只用來載入 Iris。附錄的 `check_hw1.py` 把各項計算和直接公式、SciPy 及 scikit-learn 對照，結果一致。

2 合成資料的投影（第 1 題）

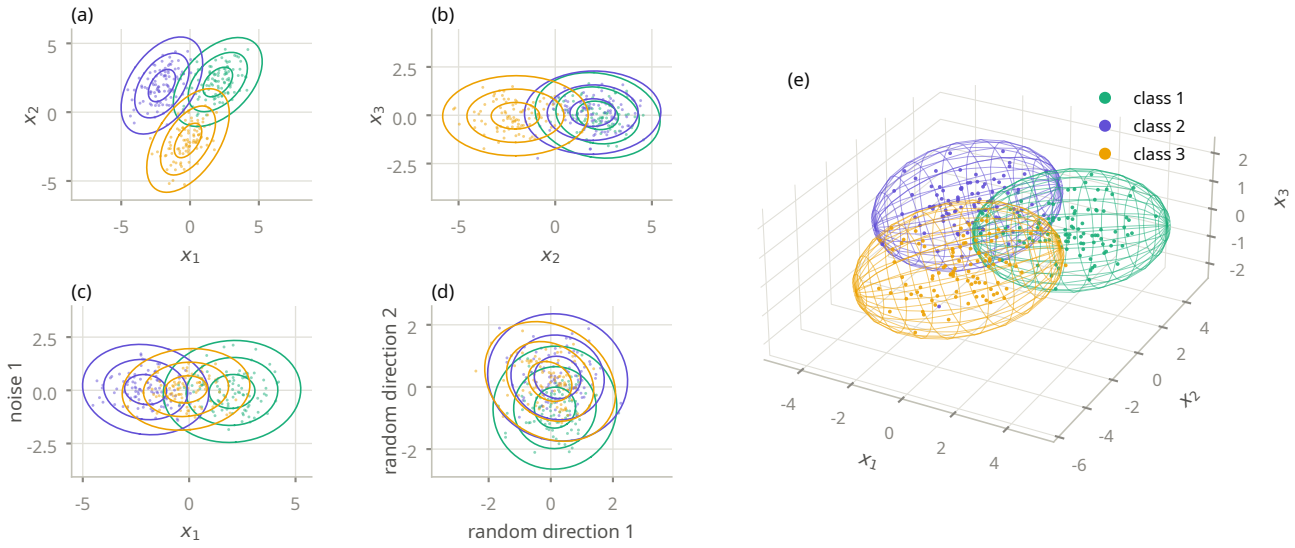


圖 1 第一次重複產生的合成資料。(a)–(d) 是四個二維投影，曲線為各類在該投影上的 Mahalanobis 距離 1、2、3；(d) 的兩個方向是 103 維空間中隨機抽出的正交單位向量。(e) 只畫前三維，橢球為各類的 95% 信賴橢球。

圖 1(a)–(d) 是同一份 103 維資料在四個平面上的投影。(a) 的 (x_1, x_2) 平面含有全部的類別資訊，三類分得很開，橢圓因 x_1 與 x_2 正相關而往右上傾斜。在 (b) 的 (x_2, x_3) 上，類別 1 與 2 的 x_2 平均都是 2，兩群重疊。x_3 在三類的分布相同，對分類沒有幫助。(c) 的雜訊軸只有三類共同的散布。(d) 是隨機方向，投影幾乎全是雜訊，三類疊在一起。圖 1(e) 是前三維與 95% 信賴橢球，也就是 Mahalanobis 距離平方不超過 $\chi^2_3(0.95) = 7.81$ 的範圍，三類落在橢球內的點分別有 99、96、93 個，接近期望的 95 個。這些都和題目的設定一致，類別資訊只在 (x_1, x_2)。

3 Hughes 現象（第 2 題）

加入 q 維雜訊後（圖 2），50/50 的 1-NN 測試準確率從 93.9% 慢慢降到 83.2%。MD 從 95.5% 很快掉到 $q = 46$ 的 39.5%，同時訓練準確率升到 100%，是典型的過度擬合。20/80 時 MD 更早，在 $q = 16$ 就掉到 47.7%，1-NN 在 $q = 100$ 則為 80.6%。

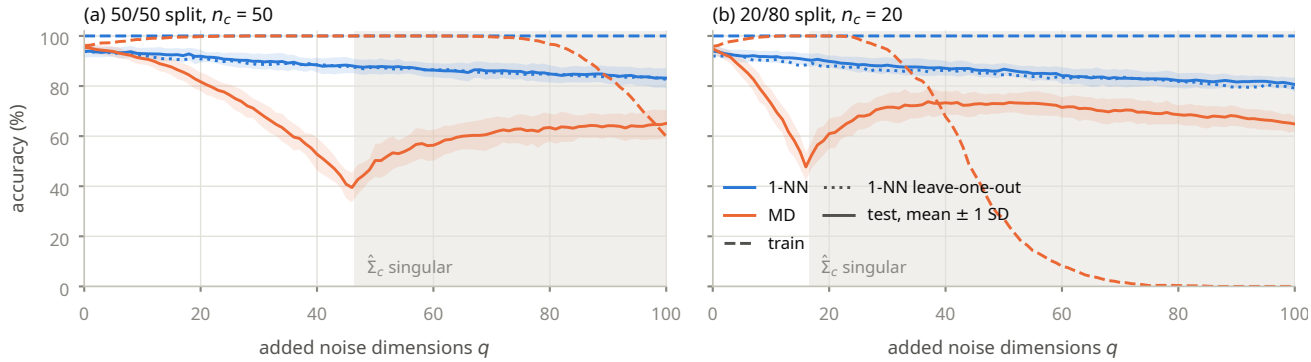


圖 2 合成資料加入 q 維雜訊後的準確率，曲線為 20 次重複的平均，色帶為 ±1 標準差。灰色區域是 $d = 3 + q \geq n_c$，各類共變異數矩陣奇異，MD 改用擬反矩陣。

這符合預期。加入的雜訊對三類都一樣，理論上最好的分類器不受影響，準確率下降完全來自用有限樣本估計參數。MD 每類要用 n_c 筆資料估計 $d(d+1)/2$ 個共變異數參數，d 接近 n_c 時最小特徵值趨近 0，取反矩陣後會把雜訊大幅放大，所以最低點正好落在最後一個可逆的維度 $d = n_c - 1$（50/50 為 $q = 46$，20/80 為 $q = 16$）。訓練樣本愈少，這個點來得愈早。1-NN 不估計參數，下降來自距離本身：$q = 100$ 時，雜訊對距離平方的貢獻平均為 100，在不同訓練點之間的起伏約為 12，已經和類別平均數之間的距離平方（16 或 20）同一量級，最近鄰愈來愈常由雜訊決定。

灰色區域裡 MD 只能用擬反矩陣，它只看各類訓練樣本所在的子空間。每個訓練點到自己類別的距離平方固定是 $n_c - 1$，到其他類別的距離卻略去了子空間以外的部分，所以訓練準確率反而下滑，20/80 在 $q \geq 86$ 時是 0%。測試準確率在這一段回升，50/50 在 $q = 100$ 回到 65.1%。$\gamma = 0.01$ 的 ridge 版本也有類似的回升（圖 3(f)），兩者都遠低於只用 3 維時的 95.5%。

4 PCA 白化與 ridge（第 3 題）

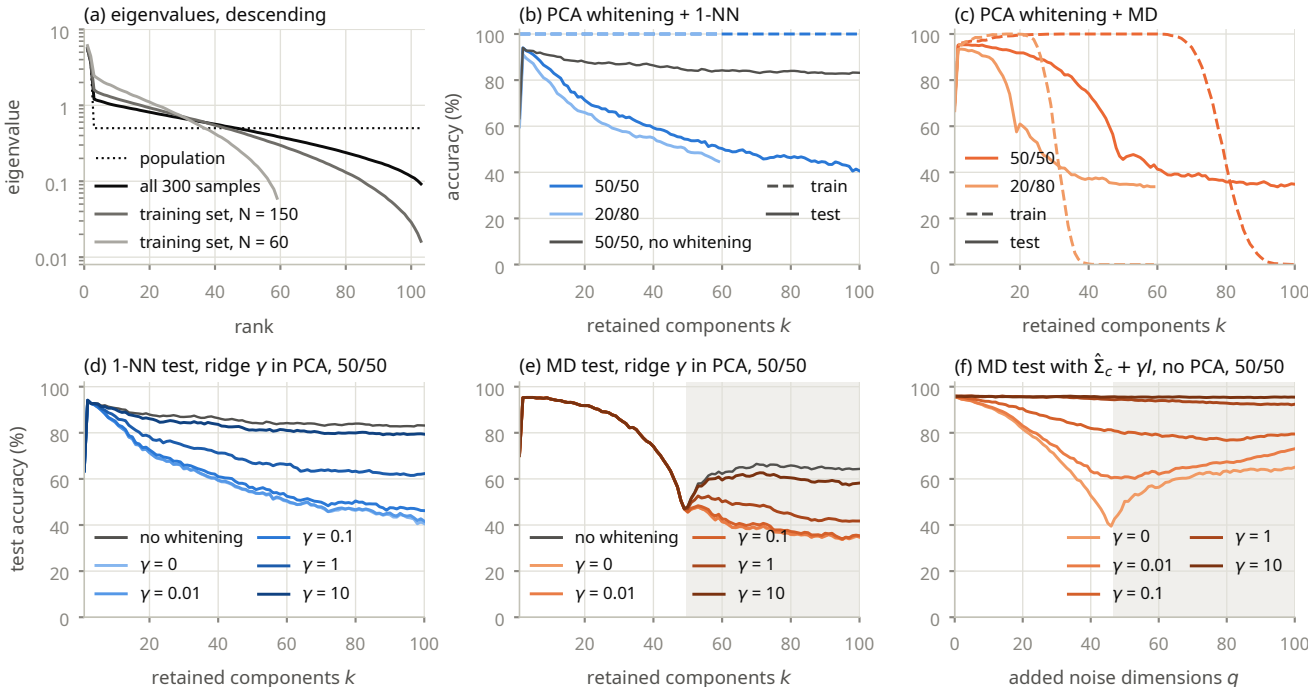


圖 3 合成資料（$q = 100$，103 維）的 PCA 實驗，曲線為 20 次重複的平均。(a) 特徵值由大到小，點線為母體值。(b)(c) $\gamma = 0$ 的 PCA 白化後保留前 k 個成分，較淺的顏色為 20/80。(d)(e) 白化時使用不同的 γ，50/50。(f) 不做 PCA，把 γI 加在 MD 的共變異數矩陣上，50/50。灰色區域表示各類共變異數矩陣奇異。

圖 3(a) 是 $q = 100$ 時 103 維資料的特徵值。母體的前兩個特徵值為 5.28 與 3.44，方向在 (x_1, x_2) 平面，其餘 101 個都是 0.5。50/50 訓練集估出的這 101 個值卻從 1.60 一路降到 0.016，樣本愈少散得愈開，20/80 的訓練集只有 59 個非零特徵值。

保留前 k 個主成分再分類（圖 3(b)(c)），準確率在 $k = 2$ 附近最高（1-NN 94.0%、MD 95.2%），之後隨 k 增加而下降，形成講義中先升後降的 Hughes 曲線。PCA 沒有用到標籤，但這份資料變異最大的兩個方向剛好是類別分開的方向，所以 $k = 2$ 就幾乎找回全部的資訊，這符合預期。k 超過 n_c 之後 MD 再次只能用擬反矩陣，訓練準

確率也和第 2 題一樣下滑。

白化對 1-NN 的傷害比我預期的大。$k = 100$ 時，不白化的 1-NN 還有 83.2%，白化後剩 40.6%。白化把每個成分除以 $\sqrt{\hat{\lambda}_j}$，而小特徵值被嚴重低估（最小 0.016，真實值 0.5），測試點在這些方向的變異數約為訓練點的 31 倍，距離幾乎由雜訊決定。Ridge 把分母改成 $\sqrt{\hat{\lambda}_j + \gamma}$，限制了放大的倍率。$k = 100$ 時 $\gamma = 0.01$、0.1、1、10 分別回到 41.7%、46.2%、62.3%、79.4%（圖 3(d)），γ 愈大愈接近不白化。$k = 2$ 時各個 γ 都在 94.0%–94.2%，幾乎沒有代價。20/80 的趨勢相同。

MD 則完全不受 PCA 中 ridge 的影響。在 $\hat{\Sigma}_c$ 可逆的 $k \le 49$，五個 γ 與不白化的曲線完全重合（圖 3(e)），因為 MD 會在轉換後重新估計共變異數，任何可逆的縮放都會被抵消。PCA 對 MD 的幫助只來自選了哪 k 個方向。若把 γI 直接加在 MD 的共變異數上，也就是講義的 $\hat{\Sigma}_c + \gamma I$（圖 3(f)），結果就不同了。$q = 100$ 時 $\gamma = 0$ 為 65.1%，$\gamma = 1$ 與 10 為 92.4% 與 95.5%，過度擬合明顯延後，和講義 ridge 與 Hughes 現象的圖一致。

5 LDA 與 rLDA（第 4 題）

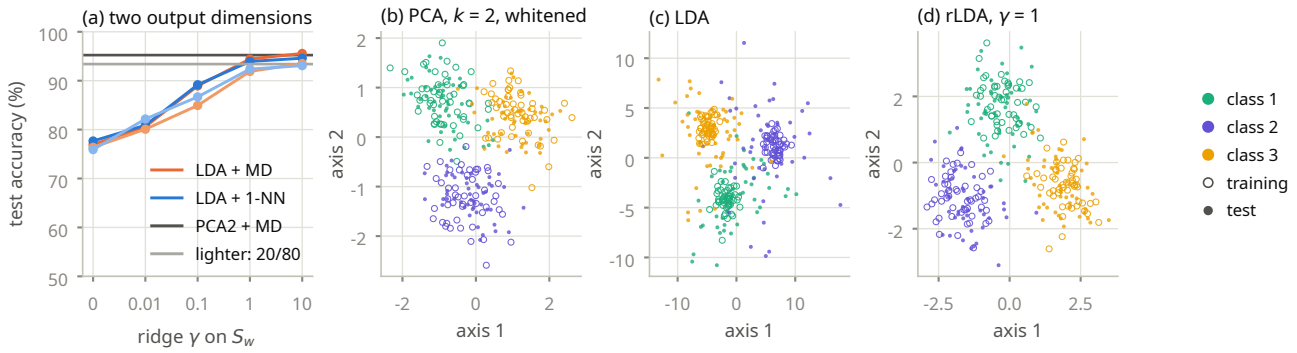


圖 4 合成資料（$q = 100$）的 LDA。(a) 取兩個 LDA 方向後的測試準確率，20 次重複的平均，水平線為 PCA2（PCA 白化取 2 維）後接 MD，較淺的線為 20/80。(b)–(d) 第一次重複、50/50 的二維投影，空心點為訓練資料，實心點為測試資料。

取兩個 LDA 方向再分類（圖 4(a)）。50/50 時不加 ridge 的 LDA 接 MD 只有 76.9%，比 PCA2 的 95.2% 低 18.3 個百分點。$\gamma = 0.01$、0.1、1、10 時升到 80.6%、88.9%、94.4%、95.6%，最後和 PCA2 相當。20/80 的趨勢相同，從 76.4% 升到 93.5%，PCA2 為 93.4%。表 1 整理了合成資料的主要結果。

表 1 合成資料的測試準確率（%，20 次重複的平均 ± 標準差）。PCA2 與 LDA 都取 2 維，再接表中的分類器。加 100 維雜訊時 MD 的共變異數奇異，使用擬反矩陣。

分割	分類器	原始 3 維	加 100 維雜訊	PCA2	LDA $\gamma = 0$	rLDA $\gamma = 1$	rLDA $\gamma = 10$
50/50	1-NN	93.9 ± 2.6	83.2 ± 4.0	94.0 ± 2.2	77.7 ± 4.9	93.9 ± 2.0	94.6 ± 2.0
50/50	MD	95.5 ± 1.9	65.1 ± 5.4	95.2 ± 2.1	76.9 ± 5.1	94.4 ± 2.0	95.6 ± 2.1
20/80	1-NN	93.9 ± 2.1	80.6 ± 2.8	91.1 ± 2.8	75.9 ± 5.5	92.4 ± 2.4	93.1 ± 2.2
20/80	MD	94.9 ± 1.8	64.9 ± 3.1	93.4 ± 1.5	76.4 ± 5.2	91.9 ± 2.7	93.5 ± 2.3

我原本以為用到標籤的 LDA 會比 PCA 好，結果相反。103 維的 S_w 只由 147 個自由度估計，最小特徵值被低估到約 0.015（真實值 0.5），LDA 於是挑中類別平均數碰巧有差異的雜訊方向。圖 4(c) 可以看得很清楚，訓練點擠成三個緊密的群（訓練準確率 100%），測試點卻散得很開。(b) 的 PCA2 與 (d) 的 rLDA 沒有這種落差。兩個 LDA 方向落在 (x_1, x_2) 平面的程度（夾角餘弦平方的平均）只有 0.16，$\gamma = 1$ 與 10 時升到 0.73 與 0.88，PCA2 為 0.90。Ridge 讓 LDA 回到真正的區分方向，這和講義說 regularized LDA 適合小樣本的說法一致。

6 Iris 資料（第 5 題）

Iris 有 150 筆、四個特徵、三類，50/50 時每類只有 25 筆訓練資料。我在四個特徵後加上 100 維 $\mathcal{N}(0, \sigma^2)$ 雜訊，σ^2 從 0.01 到 10 取 10 個值，並比較原始尺度與 z-score 標準化（只標準化四個特徵，參數取自訓練集）。依題目只用 MD 分類器。

$\sigma^2 = 0.5$ 時重做第 2 題（圖 5(a)(b)）。50/50 的測試準確率從 96.5% 降到最後一個可逆維度 $q = 20$ 的 61.8%。和合成資料不同，進入灰色區域後沒有回升，$q = 100$ 時只剩 42.0%。原始與標準化兩條曲線在可逆範圍內完全重合，10 個 σ^2 的結果在這個範圍內也完全相同。這點一開始出乎我的意料，原因和第 3 題一樣：標準化與雜訊變異數都只是可逆的縮放，MD 會把它們抵消。

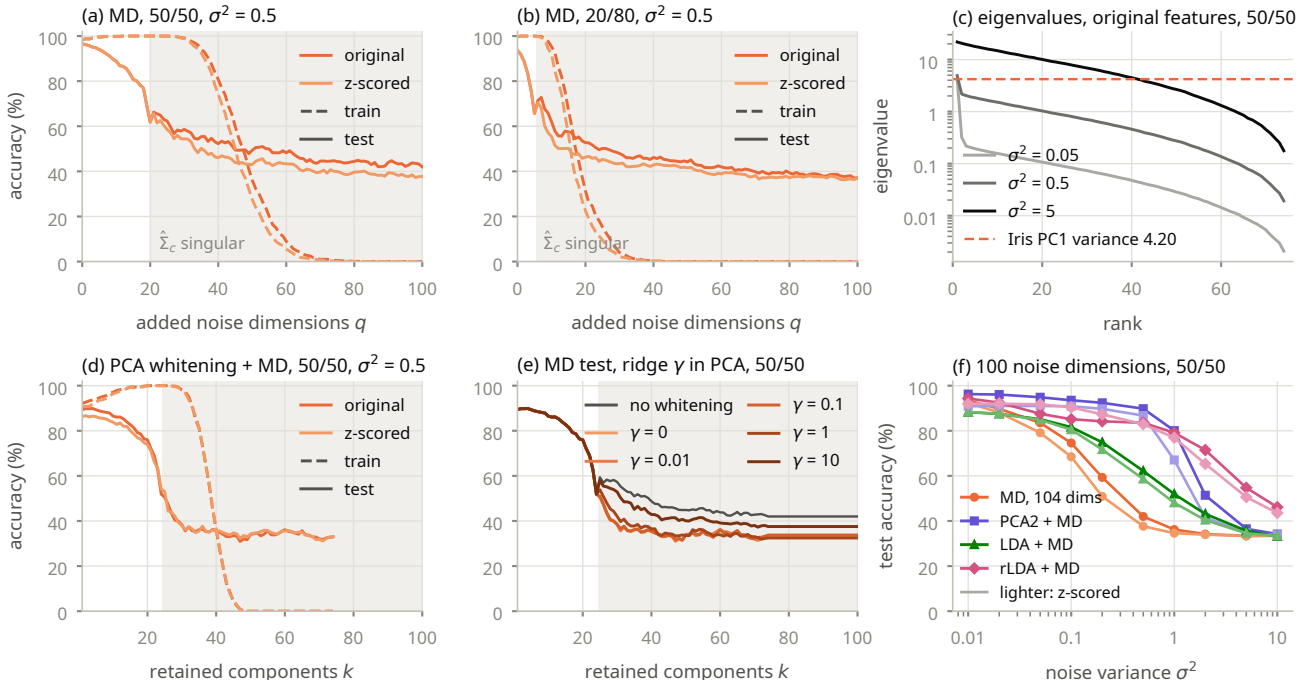


圖 5 Iris 加入 100 維 $\mathcal{N}(0,\sigma^2)$ 雜訊，分類器皆為 MD，曲線為 20 次重複的平均，較淺的顏色是四個特徵標準化後的結果。(a)(b) $\sigma^2 = 0.5$ 時逐步加入雜訊。(c) 50/50 訓練集的特徵值。(d)(e) $\sigma^2 = 0.5$、50/50 的 PCA 白化。(f) $q = 100$ 時測試準確率隨 σ^2 的變化，rLDA 的 $\gamma = 1$。灰色區域表示各類共變異數矩陣奇異。

PCA 則受尺度影響。Iris 第一主成分的變異數為 4.20，以花瓣長度為主。$\sigma^2 = 0.05$ 時它是最大的特徵值；$\sigma^2 = 5$ 時前幾個特徵值都在 17 到 22 之間，全是雜訊（圖 5(c)）。PCA2 接 MD 在 $\sigma^2 \le 0.2$ 時都有 92.4% 以上，$\sigma^2 = 1$、2、5 時降為 80.1%、51.4%、36.5%（圖 5(f)），正好對應 Iris 的方向被雜訊淹沒的過程。PCA 中不同 γ 的曲線在 $k \le 24$ 一樣完全重合（圖 5(e)）。

LDA 與 rLDA 的比較見圖 5(f) 與表 2。50/50 時 rLDA（$\gamma = 1$）在每個 σ^2 都比 LDA 好，$\sigma^2 = 0.5$ 時為 83.5% 對 62.2%，只要 $\gamma > 0$ 就高出約 20 個百分點。雜訊小時 PCA2 最好，$\sigma^2 \ge 2$ 之後 rLDA 最好（$\sigma^2 = 5$ 時 54.7%），因為它靠標籤找方向，不需要 Iris 的方向剛好變異最大。直接在 104 維上用 MD 隨 σ^2 下降得最快。20/80 時多數結果比 50/50 低，雜訊小時 LDA 反而略勝 rLDA（$\sigma^2 = 0.05$ 時 85.9% 對 82.5%）。

表 2 Iris（原始尺度，$q = 100$）的測試準確率（%，20 次重複的平均）。各方法最後都接 MD，104 維的 MD 使用擬反矩陣。

	50/50			20/80		
方法	$\sigma^2 = 0.05$	$\sigma^2 = 0.5$	$\sigma^2 = 5$	$\sigma^2 = 0.05$	$\sigma^2 = 0.5$	$\sigma^2 = 5$
MD，104 維	83.5	42.0	33.4	80.4	37.2	33.1
PCA2 + MD	94.9	89.8	36.5	94.6	82.4	36.6
LDA + MD	85.1	62.2	35.7	85.9	52.8	35.1
rLDA $\gamma = 0.1$ + MD	90.7	82.3	56.3	84.7	76.1	43.2
rLDA $\gamma = 1$ + MD	87.5	83.5	54.7	82.5	76.5	43.1
rLDA $\gamma = 10$ + MD	86.2	82.7	52.1	81.0	75.5	42.9

標準化不影響 MD 本身，對 PCA2 在雜訊小時反而不利，$\sigma^2 = 0.01$ 時從 96.3% 降到 90.9%。原始尺度下，最有區分力的花瓣長度變異數也最大（3.10），取 1 個主成分就有 93.5%，取 2 個為 96.3%。標準化後四個特徵的變異數都是 1，區分力很弱的花萼寬度（類間與類內變異數比 0.67，花瓣長度為 16.1）也被放大，第一主成分變成四個特徵的混合（係數 0.52、−0.27、0.58、0.56），取 1 或 2 個主成分都只有 90.9% 左右。

7 結果與預期

多數結果符合講義的說法。維度增加時，要估計共變異數的 MD 最早失效，1-NN 下降較慢，訓練樣本愈少，問題出現得愈早。PCA 降維與 ridge 都能延後過度擬合。和我預期不同的地方都與縮放或小樣本估計有關。白化會讓 1-NN 變差，需要夠大的 γ 才能挽回。PCA 中的 ridge、Iris 的標準化與雜訊變異數，在 $\hat{\Sigma}_c$ 可逆時完全不影響 MD。不加 ridge 的 LDA 在樣本少時比 PCA 差，因為 S_w 的反矩陣放大了雜訊方向。

附錄　程式碼

在專案根目錄執行 `uv run homework1/PR_HW1/pr_hw1.py` 會重跑全部實驗並輸出圖檔（20 個 CPU 核心約 44 秒），`uv run homework1/PR_HW1/check_hw1.py` 執行數值檢查，`uv run homework1/PR_HW1/build_report.py` 從結果檔算出報告中的每個數字並排版。套件版本由專案根目錄的 `uv.lock` 固定：Python 3.13.14、NumPy 2.5.4、SciPy 1.18.1、scikit-learn 1.9.1、Matplotlib 3.11.2。報告中的數字都由以下程式實際執行產生。

pr_hw1.py

```python
"""Pattern Recognition HW1 (NYCU, Fall 2026): experiments with linear feature spaces.

Run from the repository root (the environment is pinned by uv):
    uv run homework1/PR_HW1/pr_hw1.py               # experiments, then figures
    uv run homework1/PR_HW1/pr_hw1.py --plots-only  # figures from saved results
"""
import argparse
import json
import os
import platform
import time
from concurrent.futures import ProcessPoolExecutor
from pathlib import Path

# One BLAS thread per worker; the parallelism comes from the process pool.
for _var in ("OPENBLAS_NUM_THREADS", "OMP_NUM_THREADS", "MKL_NUM_THREADS"):
    os.environ.setdefault(_var, "1")

import numpy as np
import scipy
import sklearn
from sklearn.datasets import load_iris

HERE = Path(__file__).resolve().parent
SEED = 20261012
REPEATS = 20
FRACTIONS = (0.5, 0.2)  # share of each class used for training
N_NOISE = 100
GAMMAS = (0.0, 0.01, 0.1, 1.0, 10.0)
IRIS_VARS = (0.01, 0.02, 0.05, 0.1, 0.2, 0.5, 1.0, 2.0, 5.0, 10.0)
MEANS = np.array([[2.0, 2.0, 0.0], [-2.0, 2.0, 0.0], [0.0, -2.0, 0.0]])
SIGMA = np.array([[1.0, 0.6, 0.0], [0.6, 1.5, 0.0], [0.0, 0.0, 0.5]])
NOISE_VAR = 0.5
RTOL = 1e-12  # eigenvalues below RTOL * largest count as zero (real ones sit above 1e-10, round-off below 1e-14)


# ---------------------------------------------------------------- data

def synthetic(rep):
    """Signal (300 x 3), labels, and a 300 x 100 standard-normal noise block."""
    rng = np.random.default_rng([SEED, rep, 1])
    y = np.repeat(np.arange(3), 100)
    signal = MEANS[y] + rng.standard_normal((300, 3)) @ np.linalg.cholesky(SIGMA).T
    return signal, y, rng.standard_normal((300, N_NOISE))


def iris_noise(rep):
    return np.random.default_rng([SEED, rep, 3]).standard_normal((150, N_NOISE))


def split(y, rep, fraction):
    """Stratified split. Both fractions use the same permutation, so the 20% set lies inside the 50% set."""
    rng = np.random.default_rng([SEED, rep, 2, len(y)])
    train = [rng.permutation(np.flatnonzero(y == c)) for c in np.unique(y)]
    train = np.sort(np.concatenate([t[:round(fraction * len(t))] for t in train]))
    return train, np.setdiff1d(np.arange(len(y)), train)


def features(signal, noise, var, train, normalize=False):
    """Signal columns, z-scored with training statistics if asked, followed by noise of variance var."""
```

```
    if normalize:
        signal = (signal - signal[train].mean(0)) / signal[train].std(0)
    return np.hstack([signal, np.sqrt(var) * noise])


# ---------------------------------------------------------------- linear algebra

def cov(x):
    z = x - x.mean(0)
    return z.T @ z / len(x)


def eig_desc(s):
    """Eigenpairs of a symmetric PSD matrix, largest first, with numerical zeros set to 0."""
    lam, v = np.linalg.eigh(s)
    lam, v = lam[::-1], v[:, ::-1]
    return np.where(lam > RTOL * lam[0], lam, 0.0), v


# ---------------------------------------------------------------- classifiers

def nn_curve(xtr, ytr, xte, yte):
    """1-NN accuracy on the first 1..D columns. Columns: train, train leave-one-out, test."""
    dtr = np.zeros((len(xtr), len(xtr)))
    dte = np.zeros((len(xte), len(xtr)))
    no_self = np.diag(np.full(len(xtr), np.inf))
    out = np.empty((xtr.shape[1], 3))
    for j in range(xtr.shape[1]):
        dtr += (xtr[:, None, j] - xtr[None, :, j]) ** 2
        dte += (xte[:, None, j] - xtr[None, :, j]) ** 2
        out[j] = (np.mean(ytr[dtr.argmin(1)] == ytr),
                  np.mean(ytr[(dtr + no_self).argmin(1)] == ytr),
                  np.mean(ytr[dte.argmin(1)] == yte))
    return out


def md_fit(x, y):
    """Class means and eigendecompositions of the class covariances (ML, divided by n_c)."""
    return [(x[y == c].mean(0), *eig_desc(cov(x[y == c]))) for c in np.unique(y)]


def md_dist2(model, q, ridge=0.0):
    """Squared Mahalanobis distance to each class mean (classes x queries).

    A singular covariance uses its pseudo-inverse, i.e. directions with zero variance are ignored.
    """
    d2 = []
    for mu, lam, v in model:
        inv = 1.0 / (lam + ridge) if ridge > 0 else np.divide(1.0, lam, out=np.zeros_like(lam), where=lam > 0)
        d2.append(((q - mu) @ v) ** 2 @ inv)
    return np.array(d2)


def md_predict(model, q, ridge=0.0):
    return md_dist2(model, q, ridge).argmin(0)


def md_acc(xtr, ytr, xte, yte, ridges=(0.0,)):
    """(train, test) accuracy of the Mahalanobis classifier for each covariance ridge."""
    model = md_fit(xtr, ytr)
    q = np.vstack([xtr, xte])
    out = []
    for r in ridges:
        pred = md_predict(model, q, r)
        out.append((np.mean(pred[:len(xtr)] == ytr), np.mean(pred[len(xtr):] == yte)))
    return np.array(out)


# ---------------------------------------------------------------- PCA and LDA

```

```python
def pca_fit(x):
    return x.mean(0), *eig_desc(cov(x))


def lda_fit(x, y, gamma=0.0):
    """Solve S_b w = lambda (S_w + gamma I) w. With gamma = 0 the search stays in the range of S_w."""
    mu = x.mean(0)
    sw = np.zeros((x.shape[1], x.shape[1]))
    sb = np.zeros_like(sw)
    for c in np.unique(y):
        xc = x[y == c]
        p = len(xc) / len(x)
        sw += p * cov(xc)
        sb += p * np.outer(xc.mean(0) - mu, xc.mean(0) - mu)
    lam, v = eig_desc(sw)
    keep = lam > 0 if gamma == 0 else np.ones(len(lam), bool)
    b = v[:, keep] / np.sqrt(lam[keep] + gamma)  # b.T (S_w + gamma I) b = I
    _, u = eig_desc(b.T @ sb @ b)
    return mu, b @ u[:, :len(np.unique(y)) - 1]


def overlap(w, s):
    """Mean squared cosine between span(w) and the first s coordinate axes."""
    q, _ = np.linalg.qr(w)
    return np.sum(q[:s] ** 2) / q.shape[1]


# ------------------------------------------------------------------ experiments

def run_task(task):
    """One dataset setting, one split, one repetition: Parts 2, 3 and 4."""
    dataset, normalize, var, fraction, rep = task
    if dataset == "synthetic":
        signal, y, noise = synthetic(rep)
        informative = 2  # x3 has the same distribution in every class
    else:
        iris = load_iris()
        signal, y, noise = iris.data, iris.target, iris_noise(rep)
        informative = 4
    train, test = split(y, rep, fraction)
    x = features(signal, noise, var, train, normalize)
    xtr, ytr, xte, yte = x[train], y[train], x[test], y[test]
    s = signal.shape[1]
    with_nn = dataset == "synthetic"  # Part 5 asks for the Mahalanobis classifier only
    out = {}

    # Part 2: the s signal columns plus the first q noise columns, q = 0..100.
    if with_nn:
        out["raw_nn"] = nn_curve(xtr, ytr, xte, yte)[s - 1:]
    out["raw_md"] = np.array([md_acc(xtr[:, :s + q], ytr, xte[:, :s + q], yte, GAMMAS)
                              for q in range(N_NOISE + 1)])

    # Part 3: PCA fitted on the training set with all 100 noise columns.
    mu, lam, v = pca_fit(xtr)
    rank = np.count_nonzero(lam)
    utr, ute = (xtr - mu) @ v, (xte - mu) @ v
    utr[:, lam == 0] = 0.0  # exact zeros instead of round-off
    out["spectrum"] = lam
    pca_nn = np.full((1 + len(GAMMAS), N_NOISE, 3), np.nan)
    pca_md = np.full((1 + len(GAMMAS), N_NOISE, 2), np.nan)
    for m, gamma in enumerate((None,) + GAMMAS):  # None: plain PCA, no rescaling
        kmax = min(rank, N_NOISE) if gamma == 0 else N_NOISE  # whitening needs lambda > 0
        scale = 1.0 if gamma is None else 1.0 / np.sqrt(lam[:kmax] + gamma)
        ztr, zte = utr[:, :kmax] * scale, ute[:, :kmax] * scale
        if with_nn:
            pca_nn[m, :kmax] = nn_curve(ztr, ytr, zte, yte)
        for k in range(1, kmax + 1):
            pca_md[m, k - 1] = md_acc(ztr[:, :k], ytr, zte[:, :k], yte)[0]
    out["pca_md"] = pca_md
    if with_nn:
```

```
        out["pca_nn"] = pca_nn

    # Part 4: LDA (two Fisher directions for three classes) with ridge on S_w.
    lda_nn = np.full((len(GAMMAS), 2, 3), np.nan)
    lda_md = np.full((len(GAMMAS), 2, 2), np.nan)
    lda_overlap = np.empty(len(GAMMAS))
    for g, gamma in enumerate(GAMMAS):
        m0, w = lda_fit(xtr, ytr, gamma)
        ztr, zte = (xtr - m0) @ w, (xte - m0) @ w
        if with_nn:
            lda_nn[g] = nn_curve(ztr, ytr, zte, yte)
        for l in (1, 2):
            lda_md[g, l - 1] = md_acc(ztr[:, :l], ytr, zte[:, :l], yte)[0]
        lda_overlap[g] = overlap(w, informative)
    out["lda_md"] = lda_md
    if with_nn:
        out["lda_nn"] = lda_nn
    out["overlap"] = np.append(lda_overlap, overlap(v[:, :2], informative))  # last entry: PCA2
    return out


def tag_of(dataset, normalize, var):
    return "syn" if dataset == "synthetic" else f"iris_n{int(normalize)}_v{var:g}"


def run_all(jobs):
    tasks = [("synthetic", False, NOISE_VAR, f, r) for f in FRACTIONS for r in range(REPEATS)]
    tasks += [("iris", n, v, f, r) for n in (False, True) for v in IRIS_VARS
              for f in FRACTIONS for r in range(REPEATS)]
    with ProcessPoolExecutor(jobs) as pool:
        outs = list(pool.map(run_task, tasks, chunksize=4))
    res = {}
    for (dataset, normalize, var, fraction, rep), out in zip(tasks, outs):
        for name, arr in out.items():
            key = f"{tag_of(dataset, normalize, var)}/{name}"
            if key not in res:
                res[key] = np.full((len(FRACTIONS), REPEATS) + np.shape(arr), np.nan)
            res[key][FRACTIONS.index(fraction), rep] = arr
    return res


# ------------------------------------------------------------------ figures

TEXT_WIDTH = 6.85  # inches; figures are drawn at their printed size
INK, INK2, MUTED, GRID, AXIS, SHADE = "#0b0b0b", "#52514e", "#898781", "#e1e0d9", "#c3c2b7", "#f0efec"
# Every figure uses the same meanings: 1-NN blue, MD orange, reference lines grey, dashed = training data.
# A lighter tint marks the second condition (the 20/80 split, or z-scored Iris features).
NN, NN_TINT, MD, MD_TINT, REF, REF_TINT = "#2a78d6", "#86b6ef", "#eb6834", "#f19a64", INK2, "#a9a7a1"
NN_RAMP = ("#86b6ef", "#5598e7", "#2a78d6", "#1c5cab", "#104281")  # gamma = 0 ... 10, light to dark
MD_RAMP = ("#f19a64", "#e97c46", "#d4602a", "#a8461b", "#7a3210")
GREY_RAMP = ("#a9a7a1", "#6f6d68", INK)
CLASS_COLORS = ("#1baf7a", "#6250d6", "#eda100")
SPLIT_NAMES = ("50/50", "20/80")
TRAIN_TEST = (("train", "--"), ("test", "-"))


def style():
    import matplotlib
    matplotlib.use("Agg")
    matplotlib.rcParams.update({
        "font.family": "sans-serif", "font.sans-serif": ["Noto Sans", "DejaVu Sans"], "axes.unicode_minus": False,
        "mathtext.fontset": "dejavusans", "font.size": 7, "axes.titlesize": 7, "axes.labelsize": 7,
        "xtick.labelsize": 6.5, "ytick.labelsize": 6.5, "legend.fontsize": 6.3,
        "text.color": INK, "axes.labelcolor": INK2, "xtick.color": MUTED, "ytick.color": MUTED,
        "axes.edgecolor": AXIS, "axes.linewidth": 0.6, "axes.spines.top": False, "axes.spines.right": False,
        "axes.grid": True, "grid.color": GRID, "grid.linewidth": 0.5, "axes.axisbelow": True,
        "axes.titlelocation": "left", "axes.titlepad": 3, "lines.linewidth": 1.1,
        "legend.frameon": False, "legend.handlelength": 1.8, "legend.borderaxespad": 0.3,
        "legend.labelspacing": 0.3, "legend.columnspacing": 0.9,
        "xtick.major.size": 2, "ytick.major.size": 2, "pdf.fonttype": 42, "savefig.dpi": 300,
```

```
    })
    import matplotlib.pyplot as plt
    return plt


def save(fig, out, name):
    fig.savefig(out / f"{name}.pdf")
    fig.savefig(out / f"{name}.png", dpi=200)
    import matplotlib.pyplot as plt
    plt.close(fig)


def mean_line(ax, x, runs, color, label=None, ls="-", band=False, lw=1.1, **kw):
    """Mean over repetitions in percent; optional band of +-1 standard deviation."""
    ok = ~np.isnan(runs).all(axis=0)  # whitening without ridge stops at the PCA rank
    x, runs = np.asarray(x)[ok], 100 * runs[:, ok]
    ax.plot(x, runs.mean(0), color=color, ls=ls, lw=lw, label=label, **kw)
    if band:
        m, s = runs.mean(0), runs.std(0, ddof=1)
        ax.fill_between(x, m - s, m + s, color=color, alpha=0.13, lw=0)


def plain_log_ticks(axis):
    """Label a log axis 0.01, 0.1, 1, 10 instead of powers of ten."""
    from matplotlib.ticker import FuncFormatter
    axis.set_major_formatter(FuncFormatter(lambda t, _: f"{t:g}"))


def keys(ax, colors=(), styles=(), **kw):
    """Legend in which color and line style carry separate meanings."""
    from matplotlib.lines import Line2D
    handles = [Line2D([], [], color=c, label=l) for l, c in colors]
    handles += [Line2D([], [], color=INK2, ls=s, label=l) for l, s in styles]
    ax.legend(handles=handles, **kw)


def gamma_keys(ax, ramp, **kw):
    keys(ax, [("no whitening", REF)] + [(r"$\gamma$ = " + f"{g:g}", c) for g, c in zip(GAMMAS, ramp)], **kw)


def shade_singular(ax, start, label=False):
    """Grey band from the first dimension at which the class covariances are singular."""
    ax.axvspan(start - 0.5, 100.5, color=SHADE, lw=0, zorder=0)
    if label:
        ax.text(start + 1.5, 3, r"$\hat\Sigma_c$ singular", color=MUTED, fontsize=6.3, va="bottom")


def ellipse_2d(ax, z, color):
    """Equal-Mahalanobis-distance contours (1, 2, 3) of the class sample mean and covariance."""
    mu, (lam, v) = z.mean(0), np.linalg.eigh(cov(z))
    t = np.linspace(0, 2 * np.pi, 200)
    unit = v @ (np.sqrt(lam)[:, None] * np.vstack([np.cos(t), np.sin(t)]))
    for r in (1, 2, 3):
        ax.plot(*(mu[:, None] + r * unit), color=color, lw=0.6)


def class_keys(ax, **kw):
    from matplotlib.lines import Line2D
    ax.legend(handles=[Line2D([], [], ls="", marker="o", ms=3.5, color=c, label=f"class {i + 1}")
                       for i, c in enumerate(CLASS_COLORS)], **kw)


def fig_part1(out):
    from scipy.stats import chi2
    plt = style()
    signal, y, noise = synthetic(0)
    x = np.hstack([signal, np.sqrt(NOISE_VAR) * noise])
    basis, _ = np.linalg.qr(np.random.default_rng([SEED, 99]).standard_normal((x.shape[1], 2)))
    views = [(x[:, [0, 1]], "$x_1$", "$x_2$"), (x[:, [1, 2]], "$x_2$", "$x_3$"),
             (x[:, [0, 3]], "$x_1$", "noise 1"), (x @ basis, "random direction 1", "random direction 2")]
```

```
    fig = plt.figure(figsize=(TEXT_WIDTH, 2.95))
    outer = fig.add_gridspec(1, 2, width_ratios=(1, 1.05), wspace=0.0, left=0.065, right=1.0,
                             bottom=0.16, top=0.95)
    inner = outer[0].subgridspec(2, 2, wspace=0.5, hspace=0.62)
    for i, (z, xl, yl) in enumerate(views):
        ax = fig.add_subplot(inner[i // 2, i % 2])
        for c in range(3):
            ax.scatter(*z[y == c].T, s=1.2, color=CLASS_COLORS[c], alpha=0.5, lw=0)
            ellipse_2d(ax, z[y == c], CLASS_COLORS[c])
        ax.set_aspect("equal", adjustable="datalim")
        ax.set_xlabel(xl, labelpad=1)
        ax.set_ylabel(yl, labelpad=0)
        ax.set_title("(" + "abcd"[i] + ")")
    ax = fig.add_subplot(outer[1], projection="3d")
    r2 = chi2.ppf(0.95, 3)
    u, w = np.meshgrid(np.linspace(0, 2 * np.pi, 25), np.linspace(0, np.pi, 13))
    sphere = np.stack([np.cos(u) * np.sin(w), np.sin(u) * np.sin(w), np.cos(w)])
    inside = []
    for c in range(3):
        s = signal[y == c]
        mu, (lam, v) = s.mean(0), np.linalg.eigh(cov(s))
        shell = mu[:, None, None] + np.sqrt(r2) * np.einsum("ij,jkl->ikl", v * np.sqrt(lam), sphere)
        ax.plot_wireframe(*shell, color=CLASS_COLORS[c], lw=0.3, alpha=0.55)
        ax.scatter(*s.T, s=2, color=CLASS_COLORS[c], alpha=0.9, lw=0, depthshade=False)
        d = s - mu
        inside.append(int(np.sum(np.einsum("ij,jk,ik->i", d, np.linalg.inv(cov(s)), d) <= r2)))
    ax.set_box_aspect(np.ptp(signal, axis=0), zoom=1.1)
    ax.view_init(elev=28, azim=-62)
    ax.set_xlabel("$x_1$", labelpad=-5)
    ax.set_ylabel("$x_2$", labelpad=-5)
    ax.set_zlabel("$x_3$", labelpad=-7)
    ax.tick_params(pad=-2, labelsize=6.3)
    for axis in (ax.xaxis, ax.yaxis, ax.zaxis):
        axis.set_pane_color((1, 1, 1, 0))
        axis._axinfo["grid"].update(color=GRID, linewidth=0.4)
    ax.set_title("(e)", y=0.93, x=0.08)
    class_keys(ax, loc="upper right", bbox_to_anchor=(0.97, 0.9))
    save(fig, out, "fig1_views")
    return inside, float(r2), np.sum(basis[:2] ** 2, axis=0).tolist()


def fig_part2(res, out):
    plt = style()
    q = np.arange(N_NOISE + 1)
    fig, axes = plt.subplots(1, 2, figsize=(TEXT_WIDTH, 1.9), sharey=True)
    fig.subplots_adjust(left=0.07, right=0.99, bottom=0.2, top=0.9, wspace=0.08)
    for f, ax in enumerate(axes):
        nn, md = res["syn/raw_nn"][f], res["syn/raw_md"][f][:, :, 0]
        n_c = round(100 * FRACTIONS[f])
        shade_singular(ax, n_c - 3, label=True)
        mean_line(ax, q, nn[..., 0], NN, ls="--")
        mean_line(ax, q, nn[..., 1], NN, ls=":")
        mean_line(ax, q, nn[..., 2], NN, band=True)
        mean_line(ax, q, md[..., 0], MD, ls="--")
        mean_line(ax, q, md[..., 1], MD, band=True)
        ax.set(xlim=(0, 100), ylim=(0, 102), xlabel="added noise dimensions $q$")
        ax.set_title(f"({'ab'[f]}) {SPLIT_NAMES[f]} split, $n_c$ = {n_c}")
    axes[0].set_ylabel("accuracy (%)")
    keys(axes[1], (("1-NN", NN), ("MD", MD)),
         (("train", "--"), ("1-NN leave-one-out", ":"), (r"test, mean $\pm$ 1 SD", "-")),
         loc="lower left", bbox_to_anchor=(0.0, 0.08), ncol=2)
    save(fig, out, "fig2_hughes")


def population_spectrum():
    mu0 = MEANS.mean(0)
    sb = sum(np.outer(m - mu0, m - mu0) for m in MEANS) / 3
    top = np.linalg.eigvalsh(SIGMA[:2, :2] + sb[:2, :2])[::-1]
    return np.concatenate([top, np.full(N_NOISE + 1, NOISE_VAR)])

```

```
411
412 def fig_part3(res, out):
413     plt = style()
414     k = np.arange(1, N_NOISE + 1)
415     fig, axes = plt.subplots(2, 3, figsize=(TEXT_WIDTH, 3.6))
416     fig.subplots_adjust(left=0.07, right=0.99, bottom=0.1, top=0.95, wspace=0.28, hspace=0.5)
417     ax = axes[0, 0]
418     full = np.mean([eig_desc(cov(np.hstack([s, np.sqrt(NOISE_VAR) * n])))[0]
419                     for s, _, n in map(synthetic, range(REPEATS))], axis=0)
420     rank = np.arange(1, full.size + 1)
421     ax.semilogy(rank, population_spectrum(), color=INK, lw=0.8, ls=":", label="population")
422     ax.semilogy(rank, full, color=GREY_RAMP[2], label="all 300 samples")
423     for f, color in ((0, GREY_RAMP[1]), (1, GREY_RAMP[0])):
424         spec = res["syn/spectrum"][f].mean(0)
425         ax.semilogy(rank[spec > 0], spec[spec > 0], color=color,
426                     label=f"training set, N = {round(300 * FRACTIONS[f])}")
427     ax.set(xlabel="rank", ylabel="eigenvalue", xlim=(0, 104), ylim=(0.008, 10))
428     plain_log_ticks(ax.yaxis)
429     ax.set_title("(a) eigenvalues, descending")
430     ax.legend(loc="lower left")
431     for col, name, colors in ((1, "pca_nn", (NN, NN_TINT)), (2, "pca_md", (MD, MD_TINT))):
432         ax = axes[0, col]
433         for f in (0, 1):
434             runs = res[f"syn/{name}"][f][:, 1]  # gamma = 0
435             mean_line(ax, k, runs[..., 0], colors[f], ls="--")
436             mean_line(ax, k, runs[..., -1], colors[f])
437         ax.set(xlabel="retained components $k$", ylim=(0, 102), xlim=(1, 100))
438     mean_line(axes[0, 1], k, res["syn/pca_nn"][0][:, 0, :, 2], REF, lw=0.9)
439     axes[0, 1].set_title("(b) PCA whitening + 1-NN")
440     axes[0, 2].set_title("(c) PCA whitening + MD")
441     keys(axes[0, 1], (("50/50", NN), ("20/80", NN_TINT), ("50/50, no whitening", REF)), TRAIN_TEST,
442          loc="lower right", ncol=2)
443     keys(axes[0, 2], (("50/50", MD), ("20/80", MD_TINT)), TRAIN_TEST, loc="lower left")
444     axes[0, 1].set_ylabel("accuracy (%)")
445     for col, name, ramp in ((0, "pca_nn", NN_RAMP), (1, "pca_md", MD_RAMP)):
446         ax = axes[1, col]
447         runs = res[f"syn/{name}"][0]
448         mean_line(ax, k, runs[:, 0, :, -1], REF, lw=0.9)
449         for g, color in enumerate(ramp):
450             mean_line(ax, k, runs[:, g + 1, :, -1], color)
451         ax.set(xlabel="retained components $k$", ylim=(0, 102), xlim=(1, 100))
452         gamma_keys(ax, ramp, loc="lower left", ncol=2)
453     axes[1, 0].set_title(r"(d) 1-NN test, ridge $\gamma$ in PCA, 50/50")
454     axes[1, 1].set_title(r"(e) MD test, ridge $\gamma$ in PCA, 50/50")
455     axes[1, 0].set_ylabel("test accuracy (%)")
456     shade_singular(axes[1, 1], 50)
457     ax = axes[1, 2]
458     q = np.arange(N_NOISE + 1)
459     shade_singular(ax, 47)
460     for g, color in enumerate(MD_RAMP):
461         mean_line(ax, q, res["syn/raw_md"][0][:, :, g, 1], color)
462     ax.set(xlabel="added noise dimensions $q$", ylim=(0, 102), xlim=(0, 100))
463     ax.set_title(r"(f) MD test with $\hat\Sigma_c+\gamma I$, no PCA, 50/50")
464     keys(ax, [(r"$\gamma$ = " + f"{g:g}", c) for g, c in zip(GAMMAS, MD_RAMP)], loc="lower right", ncol=2)
465     save(fig, out, "fig3_pca")
466
467
468 def fig_part4(res, out):
469     plt = style()
470     fig = plt.figure(figsize=(TEXT_WIDTH, 1.9))
471     grid = fig.add_gridspec(1, 5, width_ratios=(1.15, 1, 1, 1, 0.6), wspace=0.36,
472                             left=0.065, right=0.995, bottom=0.21, top=0.88)
473     ax = fig.add_subplot(grid[0])
474     pos = np.arange(len(GAMMAS))
475     for f, (nn_c, md_c, ref_c) in enumerate(((NN, MD, REF), (NN_TINT, MD_TINT, REF_TINT))):
476         ax.axhline(100 * res["syn/pca_md"][f][:, 1, 1, 1].mean(), color=ref_c, lw=1.0)
477         ax.plot(pos, 100 * res["syn/lda_md"][f][:, :, 1, 1].mean(0), color=md_c, marker="o", ms=2.5)
478         ax.plot(pos, 100 * res["syn/lda_nn"][f][:, :, 1, 2].mean(0), color=nn_c, marker="o", ms=2.5)
479     ax.set_xticks(pos, [f"{g:g}" for g in GAMMAS])
480     ax.set(xlabel=r"ridge $\gamma$ on $S_w$", ylabel="test accuracy (%)", ylim=(50, 100))
```

```
    ax.set_title("(a) two output dimensions")
    keys(ax, (("LDA + MD", MD), ("LDA + 1-NN", NN), ("PCA2 + MD", REF), ("lighter: 20/80", REF_TINT)),
         loc="lower right")
    signal, y, noise = synthetic(0)
    train, test = split(y, 0, 0.5)
    x = features(signal, noise, NOISE_VAR, train)
    mu, lam, v = pca_fit(x[train])
    maps = [(r"(b) PCA, $k$ = 2, whitened", lambda z: (z - mu) @ v[:, :2] / np.sqrt(lam[:2]))]
    for title, gamma in (("(c) LDA", 0.0), (r"(d) rLDA, $\gamma$ = 1", 1.0)):
        m0, w = lda_fit(x[train], y[train], gamma)
        maps.append((title, lambda z, m0=m0, w=w: (z - m0) @ w))
    for i, (title, f) in enumerate(maps, start=1):
        ax = fig.add_subplot(grid[i])
        for c in range(3):
            ztr, zte = f(x[train][y[train] == c]), f(x[test][y[test] == c])
            ax.scatter(*zte.T, s=2.2, color=CLASS_COLORS[c], alpha=0.8, lw=0)
            ax.scatter(*ztr.T, s=5, facecolors="none", edgecolors=CLASS_COLORS[c], lw=0.4, alpha=0.9)
        ax.set_title(title)
        ax.set_xlabel("axis 1", labelpad=1)
        ax.set_ylabel("axis 2", labelpad=1)
    from matplotlib.lines import Line2D
    ax = fig.add_subplot(grid[4])
    ax.axis("off")
    handles = [Line2D([], [], ls="", marker="o", ms=3.5, color=c, label=f"class {i + 1}")
               for i, c in enumerate(CLASS_COLORS)]
    handles += [Line2D([], [], ls="", marker="o", ms=3.5, mfc="none", mec=INK2, mew=0.6, label="training"),
                Line2D([], [], ls="", marker="o", ms=2.5, color=INK2, label="test")]
    ax.legend(handles=handles, loc="center left", borderaxespad=0, handletextpad=0.2)
    save(fig, out, "fig4_lda")


def fig_part5(res, out):
    plt = style()
    fig, axes = plt.subplots(2, 3, figsize=(TEXT_WIDTH, 3.55))
    fig.subplots_adjust(left=0.07, right=0.99, bottom=0.1, top=0.95, wspace=0.28, hspace=0.52)
    q, k = np.arange(N_NOISE + 1), np.arange(1, N_NOISE + 1)
    norm_keys = (("original", MD), ("z-scored", MD_TINT))
    for f, ax in enumerate(axes[0, :2]):
        shade_singular(ax, round(50 * FRACTIONS[f]) - 4, label=True)
        for n, color in ((0, MD), (1, MD_TINT)):
            runs = res[f"iris_n{n}_v0.5/raw_md"][f][:, :, 0]
            mean_line(ax, q, runs[..., 0], color, ls="--")
            mean_line(ax, q, runs[..., 1], color)
        ax.set(xlabel="added noise dimensions $q$", ylim=(0, 102), xlim=(0, 100))
        ax.set_title(f"({'ab'[f]}) MD, {SPLIT_NAMES[f]}, " + r"$\sigma^2$ = 0.5")
        keys(ax, norm_keys, TRAIN_TEST, loc="upper right")
    axes[0, 0].set_ylabel("accuracy (%)")
    ax = axes[0, 2]
    for var, color in zip((0.05, 0.5, 5.0), GREY_RAMP):
        spec = res[f"iris_n0_v{var:g}/spectrum"][0].mean(0)
        r = np.arange(1, spec.size + 1)
        ax.semilogy(r[spec > 0], spec[spec > 0], color=color, label=r"$\sigma^2$ = " + f"{var:g}")
    iris_var = np.linalg.eigvalsh(cov(load_iris().data))[-1]
    ax.axhline(iris_var, color=MD, lw=0.9, ls=(0, (4, 2)), label=f"Iris PC1 variance {iris_var:.2f}")
    ax.set(xlabel="rank", ylabel="eigenvalue", xlim=(0, 76))
    plain_log_ticks(ax.yaxis)
    ax.set_title("(c) eigenvalues, original features, 50/50")
    ax.legend(loc="lower left")
    ax = axes[1, 0]
    shade_singular(ax, 25)
    for n, color in ((0, MD), (1, MD_TINT)):
        runs = res[f"iris_n{n}_v0.5/pca_md"][0][:, 1]
        mean_line(ax, k, runs[..., 0], color, ls="--")
        mean_line(ax, k, runs[..., 1], color)
    ax.set(xlabel="retained components $k$", ylabel="accuracy (%)", ylim=(0, 102), xlim=(1, 100))
    ax.set_title(r"(d) PCA whitening + MD, 50/50, $\sigma^2$ = 0.5")
    keys(ax, norm_keys, TRAIN_TEST, loc="upper right")
    ax = axes[1, 1]
    shade_singular(ax, 25)
    runs = res["iris_n0_v0.5/pca_md"][0]
```

```
    mean_line(ax, k, runs[:, 0, :, 1], REF, lw=0.9)
    for g, color in enumerate(MD_RAMP):
        mean_line(ax, k, runs[:, g + 1, :, 1], color)
    ax.set(xlabel="retained components $k$", ylim=(0, 102), xlim=(1, 100))
    ax.set_title(r"(e) MD test, ridge $\gamma$ in PCA, 50/50")
    gamma_keys(ax, MD_RAMP, loc="upper right", ncol=2)
    ax = axes[1, 2]
    v = np.array(IRIS_VARS)
    curves = ((("#eb6834", "#f19a64"), "o", "MD, 104 dims", "raw_md", lambda a: a[:, 100, 0, 1]),
              (("#6250d6", "#a39be8"), "s", "PCA2 + MD", "pca_md", lambda a: a[:, 1, 1, 1]),
              (("#008300", "#6fb86f"), "^", "LDA + MD", "lda_md", lambda a: a[:, 0, 1, 1]),
              (("#d55181", "#e99ab8"), "D", "rLDA + MD", "lda_md", lambda a: a[:, 3, 1, 1]))  # rLDA: gamma = 1
    from matplotlib.lines import Line2D
    handles = []
    for (dark, light), marker, name, key, pick in curves:
        for n, color in ((0, dark), (1, light)):
            acc = [100 * pick(res[f"iris_n{n}_v{s:g}/{key}"][0]).mean() for s in v]
            ax.semilogx(v, acc, color=color, marker=marker, ms=2.4, lw=1.0)
        handles.append(Line2D([], [], color=dark, marker=marker, ms=2.4, label=name))
    handles.append(Line2D([], [], color=REF_TINT, lw=1.0, label="lighter: z-scored"))
    ax.set(xlabel=r"noise variance $\sigma^2$", ylabel="test accuracy (%)", ylim=(0, 102))
    plain_log_ticks(ax.xaxis)
    ax.set_title("(f) 100 noise dimensions, 50/50")
    ax.legend(handles=handles, loc="lower left", fontsize=6.0, handlelength=1.6, labelspacing=0.12, borderaxespad=0.15)
    save(fig, out, "fig5_iris")


def make_figures(out):
    res = dict(np.load(out / "results.npz"))
    figdir = out / "figures"
    figdir.mkdir(exist_ok=True)
    inside, r2, random_inplane = fig_part1(figdir)
    fig_part2(res, figdir)
    fig_part3(res, figdir)
    fig_part4(res, figdir)
    fig_part5(res, figdir)
    (out / "part1.json").write_text(json.dumps(dict(inside_95=inside, chi2_3_95=r2,
                                                    random_directions_in_x1x2=random_inplane), indent=2) + "\n")


def main():
    parser = argparse.ArgumentParser(description=__doc__.splitlines()[0])
    parser.add_argument("--out", type=Path, default=HERE / "results")
    parser.add_argument("--jobs", type=int, default=os.cpu_count())
    parser.add_argument("--plots-only", action="store_true")
    args = parser.parse_args()
    args.out.mkdir(parents=True, exist_ok=True)
    if not args.plots_only:
        start = time.perf_counter()
        res = run_all(args.jobs)
        np.savez_compressed(args.out / "results.npz", **res)
        info = dict(seed=SEED, repeats=REPEATS, fractions=FRACTIONS, gammas=GAMMAS,
                    iris_noise_variances=IRIS_VARS, synthetic_noise_variance=NOISE_VAR,
                    eigenvalue_rtol=RTOL, seconds=round(time.perf_counter() - start, 1),
                    python=platform.python_version(), numpy=np.__version__,
                    scipy=scipy.__version__, scikit_learn=sklearn.__version__)
        (args.out / "run_info.json").write_text(json.dumps(info, indent=2) + "\n")
        print(f"experiments done in {info['seconds']} s")
    make_figures(args.out)
    print(f"figures written to {args.out / 'figures'}")


if __name__ == "__main__":
    main()
```

check_hw1.py

```
"""Checks pr_hw1.py against direct formulas, SciPy and scikit-learn.

Run from the repository root:  uv run homework1/PR_HW1/check_hw1.py
```

```
"""
import numpy as np
from scipy.linalg import eigh, subspace_angles
from sklearn.neighbors import KNeighborsClassifier

import pr_hw1 as hw

rng = np.random.default_rng(7)
y3 = lambda n: np.repeat(np.arange(3), n)


def direct_md2(x, y, q, ridge=0.0):
    out = []
    for c in range(3):
        xc = x[y == c]
        s = np.cov(xc, rowvar=False, bias=True) + ridge * np.eye(x.shape[1])
        inv = np.linalg.pinv(s, rcond=hw.RTOL, hermitian=True)  # inv(s) when s is invertible
        out.append(np.einsum("ij,jk,ik->i", q - xc.mean(0), inv, q - xc.mean(0)))
    return np.array(out)


# Mahalanobis distances: invertible, singular, and with a covariance ridge.
for n, d in ((40, 5), (40, 60), (12, 30)):
    x, q = rng.normal(size=(3 * n, d)), rng.normal(size=(50, d))
    for ridge in (0.0, 0.1):
        got, ref = hw.md_dist2(hw.md_fit(x, y3(n)), q, ridge), direct_md2(x, y3(n), q, ridge)
        assert np.allclose(got, ref, rtol=1e-6, atol=1e-8), (n, d, ridge)
print("Mahalanobis distance matches pinv formula: ok")

# Invertible case: predictions do not change under x -> A x + b.
x, q = rng.normal(size=(120, 6)), rng.normal(size=(80, 6))
a, b = rng.normal(size=(6, 6)) + 3 * np.eye(6), rng.normal(size=6)
p1 = hw.md_predict(hw.md_fit(x, y3(40)), q)
p2 = hw.md_predict(hw.md_fit(x @ a.T + b, y3(40)), q @ a.T + b)
assert np.array_equal(p1, p2)
print("affine invariance of the Mahalanobis classifier: ok")

# Singular case: every training point sits at squared distance n_c - 1 from its own class mean.
x = rng.normal(size=(60, 50))
model = hw.md_fit(x, y3(20))
own = hw.md_dist2(model, x)[y3(20), np.arange(60)]
assert np.allclose(own, 19.0)
print("pinv distance of a training point to its own class = n_c - 1: ok")

# Ranks: class covariance n_c - 1, PCA covariance N - 1, within-class scatter N - C.
x = rng.normal(size=(60, 103))
assert np.count_nonzero(hw.eig_desc(hw.cov(x[:20]))[0]) == 19
assert np.count_nonzero(hw.pca_fit(x)[1]) == 59
sw = sum(hw.cov(x[y3(20) == c]) / 3 for c in range(3))
assert np.count_nonzero(hw.eig_desc(sw)[0]) == 57
print("ranks 19 / 59 / 57 for n_c = 20, N = 60, d = 103: ok")

# PCA whitening: identity covariance; with ridge, diag(lambda / (lambda + gamma)).
x = rng.normal(size=(200, 8)) @ rng.normal(size=(8, 8))
mu, lam, v = hw.pca_fit(x)
for gamma in (0.0, 0.5):
    z = (x - mu) @ v / np.sqrt(lam + gamma)
    assert np.allclose(hw.cov(z), np.diag(lam / (lam + gamma)), atol=1e-10)
print("PCA whitening covariance with and without ridge: ok")

# LDA: same subspace as SciPy's generalized symmetric eigensolver.
x = rng.normal(size=(150, 10)) + np.repeat(rng.normal(size=(3, 10)), 50, axis=0)
y = y3(50)
sw = sum(hw.cov(x[y == c]) / 3 for c in range(3))
sb = sum(np.outer(x[y == c].mean(0) - x.mean(0), x[y == c].mean(0) - x.mean(0)) / 3 for c in range(3))
for gamma in (0.0, 0.1, 1.0):
    _, w = hw.lda_fit(x, y, gamma)
    _, vecs = eigh(sb, sw + gamma * np.eye(10))
    assert np.max(subspace_angles(w, vecs[:, -2:])) < 1e-8
print("LDA subspace matches scipy.linalg.eigh(S_b, S_w + gamma I): ok")
```

```

# Singular S_w (more dimensions than N - C): gamma = 0 must match the pseudo-inverse solution.
x = rng.normal(size=(30, 40)) + np.repeat(rng.normal(size=(3, 40)), 10, axis=0)
y = y3(10)
sw = sum(hw.cov(x[y == c]) / 3 for c in range(3))
sb = sum(np.outer(x[y == c].mean(0) - x.mean(0), x[y == c].mean(0) - x.mean(0)) / 3 for c in range(3))
vals, vecs = np.linalg.eig(np.linalg.pinv(sw, rcond=hw.RTOL, hermitian=True) @ sb)
top = vecs[:, np.argsort(vals.real)[::-1][:2]].real
assert np.max(subspace_angles(hw.lda_fit(x, y, 0.0)[1], top)) < 1e-6
print("LDA with singular S_w matches the pseudo-inverse solution: ok")

# 1-NN on every prefix of the columns, against scikit-learn.
x, q = rng.normal(size=(90, 7)), rng.normal(size=(40, 7))
yq = rng.integers(0, 3, 40)
curve = hw.nn_curve(x, y3(30), q, yq)
for k in range(1, 8):
    knn = KNeighborsClassifier(n_neighbors=1).fit(x[:, :k], y3(30))
    assert curve[k - 1, 0] == 1.0
    assert curve[k - 1, 2] == knn.score(q[:, :k], yq)
    d = ((x[:, None, :k] - x[None, :, :k]) ** 2).sum(-1) + np.diag(np.full(90, np.inf))
    assert curve[k - 1, 1] == np.mean(y3(30)[d.argmin(1)] == y3(30))
print("1-NN prefix curves and leave-one-out match direct computation: ok")

# Splits: stratified, disjoint, nested; normalization uses training rows only.
signal, y, noise = hw.synthetic(0)
tr50, te50 = hw.split(y, 0, 0.5)
tr20, te20 = hw.split(y, 0, 0.2)
assert np.bincount(y[tr50]).tolist() == [50] * 3 and np.bincount(y[tr20]).tolist() == [20] * 3
assert not np.intersect1d(tr50, te50).size and set(tr20) <= set(tr50)
moved = signal.copy()
moved[te50] += 1e3
assert np.array_equal(hw.features(signal, noise, 0.5, tr50, True)[tr50],
                      hw.features(moved, noise, 0.5, tr50, True)[tr50])
print("stratified nested splits, training-only normalization: ok")

# Generator: class means and covariance pooled over the 20 repetitions.
data = [hw.synthetic(r) for r in range(hw.REPEATS)]
sig = np.vstack([d[0] for d in data])
lab = np.concatenate([d[1] for d in data])
noise = np.vstack([d[2] for d in data]) * np.sqrt(hw.NOISE_VAR)
assert np.abs(np.array([sig[lab == c].mean(0) for c in range(3)]) - hw.MEANS).max() < 0.1
assert np.abs(sum(hw.cov(sig[lab == c]) for c in range(3)) / 3 - hw.SIGMA).max() < 0.1
assert abs(noise.var() - hw.NOISE_VAR) < 0.01
print("generator moments (6000 samples): ok")
```

build_report.py

```
"""Compute every number quoted in the report from results/, fill report/template.tex, and typeset it.

Run from the repository root after pr_hw1.py:  uv run homework1/PR_HW1/build_report.py
Always writes the public PDF, which has no student ID. If student.local.json exists, it first writes the
e3 copy with the ID on page 1. The asserts check the qualitative sentences of the report.
"""
import json
import re
import shutil
import subprocess
from pathlib import Path

import numpy as np
from sklearn.datasets import load_iris

import pr_hw1 as hw

HERE = Path(__file__).resolve().parent
REPORT = HERE / "report"
PUBLIC_PDF = HERE.parent / "public" / "PR_HW1_蔡秀吉.pdf"  # pushed to GitHub
PRIVATE = HERE / "student.local.json"  # {"student_id": "..."}; ignored by git, never committed
R = dict(np.load(HERE / "results" / "results.npz"))
P1 = json.loads((HERE / "results" / "part1.json").read_text())
```

```
INFO = json.loads((HERE / "results" / "run_info.json").read_text())
G = {g: i for i, g in enumerate(hw.GAMMAS)}
GNAME = {0.0: "0", 0.01: "001", 0.1: "01", 1.0: "1", 10.0: "10"}
T50, T20 = 0, 1
v = {}


def pct(a):
    return 100 * np.nanmean(a)


def f1(x):
    return f"{x:.1f}"


def pm(a):
    return f"{100 * a.mean():.1f} $\\pm$ {100 * a.std(ddof=1):.1f}"


# Part 1
v["in1"], v["in2"], v["in3"] = map(str, P1["inside_95"])
v["chi3"] = f"{P1['chi2_3_95']:.2f}"

# Part 2: raw features, q = 0..100 noise columns
nn, md = R["syn/raw_nn"], R["syn/raw_md"]
for f, t in ((T50, "50"), (T20, "20")):
    te = md[f][:, :, 0, 1].mean(0)
    v[f"mdmin_{t}"], v[f"mdargmin_{t}"] = f1(100 * te.min()), str(te.argmin())
    assert te.argmin() == round(100 * hw.FRACTIONS[f]) - 4  # lowest at the last invertible dimension d = n_c - 1
    v[f"nnq100_{t}"] = f1(pct(nn[f][:, 100, 2]))
te50 = md[T50][:, :, 0, 1].mean(0)
assert md[T50][:, 46, 0, 0].mean() == 1, "50/50: MD fits the training set perfectly at its minimum"
v["nnq0_50"], v["mdq0_50"], v["mdq100_50"] = f1(pct(nn[T50][:, 0, 2])), f1(100 * te50[0]), f1(100 * te50[100])
tr20 = md[T20][:, :, 0, 0].mean(0)
zero_from = int(np.flatnonzero(tr20 == 0)[0])
assert np.all(tr20[zero_from:] == 0)
v["mdtrzero_20"] = str(zero_from)
ridge001 = md[T50][:, :, G[0.01], 1].mean(0)
assert ridge001[100] > ridge001[47:].min() + 0.05, "gamma = 0.01 also rebounds in the singular region"
assert max(te50[100], ridge001[100]) < te50[0] - 0.1, "both stay far below the 3-dimensional accuracy"
v["noisesd"] = f"{np.sqrt(1.5 * 100):.0f}"  # SD over training points of a sum of 100 terms (n_test - n_train)^2

# Part 3: PCA on the 103-dimensional data
spec50 = R["syn/spectrum"][T50].mean(0)
pop = hw.population_spectrum()
v["pop1"], v["pop2"] = f"{pop[0]:.2f}", f"{pop[1]:.2f}"
v["s3"], v["slast"] = f"{spec50[2]:.2f}", f"{spec50[-1]:.3f}"
v["amp"] = f"{0.5 / spec50[-1]:.0f}"
v["rank20"] = str(int(np.count_nonzero(R["syn/spectrum"][T20].mean(0))))
pnn, pmd = R["syn/pca_nn"], R["syn/pca_md"]
v["pnn2"], v["pmd2"] = f1(pct(pnn[T50][:, 1, 1, 2])), f1(pct(pmd[T50][:, 1, 1, 1]))
assert pnn[T50][:, 1, :, 2].mean(0).argmax() == 1, "1-NN after whitening peaks at k = 2"
plateau = pmd[T50][:, 1, 1:5, 1].mean(0)
assert plateau.max() - plateau.min() < 0.005 and pmd[T50][:, 1, :, 1].mean(0).argmax() <= 4
assert pct(pmd[T50][:, 1, 99, 0]) < 1, "MD training accuracy collapses again beyond n_c"
v["pnnplain100"] = f1(pct(pnn[T50][:, 0, 99, 2]))
for g in hw.GAMMAS:
    v[f"pnn100_{GNAME[g]}"] = f1(pct(pnn[T50][:, 1 + G[g], 99, 2]))
for f in (T50, T20):  # 1-NN at k = 100: larger gamma moves toward no whitening, in both splits
    ladder = [pct(pnn[f][:, 1 + G[g], 99, 2]) for g in hw.GAMMAS[1:]] + [pct(pnn[f][:, 0, 99, 2])]
    assert np.all(np.diff(ladder) > 0)
v["pnnk2lo"] = f1(min(pct(pnn[T50][:, m, 1, 2]) for m in range(6)))
v["pnnk2hi"] = f1(max(pct(pnn[T50][:, m, 1, 2]) for m in range(6)))
for f, n_c in ((T50, 50), (T20, 20)):
    sub = pmd[f][:, :, :n_c - 1, 1]  # every PCA scaling, k = 1 .. n_c - 1
    assert np.all(sub == sub[:, :1]), "MD after PCA ignores the scaling while invertible"
for g in (0.0, 1.0, 10.0):
    v[f"ridge100_{GNAME[g]}"] = f1(pct(md[T50][:, 100, G[g], 1]))
assert np.all(np.diff([pct(md[T50][:, 100, G[g], 1]) for g in hw.GAMMAS]) > 0), "covariance ridge helps MD"

```

```
# Part 4: LDA
lmd, ov = R["syn/lda_md"], R["syn/overlap"]
for g in hw.GAMMAS:
    v[f"lda_{GNAME[g]}"] = f1(pct(lmd[T50][:, G[g], 1, 1]))
assert abs(pct(lmd[T50][:, G[10.0], 1, 1]) - pct(pmd[T50][:, 1, 1, 1])) < 1, "rLDA(10) matches PCA2"
assert pct(lmd[T50][:, 0, 1, 0]) == 100, "LDA without ridge fits the training set perfectly"
v["ldagap"] = f1(pct(pmd[T50][:, 1, 1, 1]) - pct(lmd[T50][:, 0, 1, 1]))
v["lda20_0"], v["lda20_10"] = f1(pct(lmd[T20][:, 0, 1, 1])), f1(pct(lmd[T20][:, 4, 1, 1]))
v["pmd2_20"] = f1(pct(pmd[T20][:, 1, 1, 1]))
for g in (0.0, 1.0, 10.0):
    v[f"ov_{GNAME[g]}"] = f"{ov[T50][:, G[g]].mean():.2f}"
v["ovpca"] = f"{ov[T50][:, 5].mean():.2f}"
swmin = []
for rep in range(hw.REPEATS):
    s, y, n = hw.synthetic(rep)
    tr, _ = hw.split(y, rep, 0.5)
    x, yt = hw.features(s, n, hw.NOISE_VAR, tr)[tr], y[tr]
    swmin.append(hw.eig_desc(sum(np.mean(yt == c) * hw.cov(x[yt == c]) for c in range(3)))[0][-1])
v["swmin"] = f"{np.mean(swmin):.3f}"
rows = []
for f, split_name in ((T50, "50/50"), (T20, "20/80")):
    for raw, pca, lda, clf, label in ((nn[f], pnn[f][:, 1, 1], R["syn/lda_nn"][f][:, :, 1], 2, "1-NN"),
                                      (md[f][:, :, 0], pmd[f][:, 1, 1], lmd[f][:, :, 1], 1, "MD")):
        cells = [pm(raw[:, 0, clf]), pm(raw[:, 100, clf]), pm(pca[:, clf])]
        cells += [pm(lda[:, G[g], clf]) for g in (0.0, 1.0, 10.0)]
        rows.append(f"{split_name} & {label} & " + " & ".join(cells) + r" \\")
v["table1"] = "\n".join(rows)

# Part 5: Iris with 100 noise dimensions
iris, target = load_iris(return_X_y=True)
I = lambda n, s, key: R[f"iris_n{n}_v{s:g}/{key}"]
MD100, PCA1, PCA2 = (lambda a: a[:, 100, 0, 1]), (lambda a: a[:, 1, 0, 1]), (lambda a: a[:, 1, 1, 1])
LDA = lambda g: lambda a: a[:, G[g], 1, 1]
acc = lambda n, s, key, pick, f=T50: pct(pick(I(n, s, key)[f]))
te = I(0, 0.5, "raw_md")[T50][:, :, 0, 1].mean(0)
v["iq0_50"], v["iqlast_50"], v["iqarg_50"], v["iq100_o"] = f1(100 * te[0]), f1(100 * te[20]), "20", f1(100 * te[100])
assert te[100] == te[21:].min() and te[100] < te[20], "Iris does not recover in the singular region"
for f, n_c in ((T50, 25), (T20, 10)):
    stack = np.stack([I(n, s, "raw_md")[f][:, :n_c - 4, 0, 1] for n in (0, 1) for s in hw.IRIS_VARS])
    assert np.all(stack == stack[:1]), "invertible MD ignores the scaling of features and noise"
assert np.all(I(0, 0.5, "pca_md")[T50][:, :, :24, 1] == I(0, 0.5, "pca_md")[T50][:, :1, :24, 1])
v["irispc1"] = f"{np.linalg.eigvalsh(hw.cov(iris))[-1]:.2f}"
sp5 = I(0, 5.0, "spectrum")[T50].mean(0)
v["isp5lo"], v["isp5hi"] = f"{sp5[5]:.0f}", f"{sp5[0]:.0f}"
assert sp5[5] > float(v["irispc1"]), "with sigma2 = 5 the leading eigenvalues are all noise"
v["ipcamin02"] = f1(min(acc(0, s, "pca_md", PCA2) for s in hw.IRIS_VARS if s <= 0.2))
for s, key in ((0.01, "001"), (1.0, "1"), (2.0, "2"), (5.0, "5")):
    v[f"ipca_{key}"] = f1(acc(0, s, "pca_md", PCA2))
v["ilda_05"], v["irlda_05"] = f1(acc(0, 0.5, "lda_md", LDA(0.0))), f1(acc(0, 0.5, "lda_md", LDA(1.0)))
v["irlda_5"] = f1(acc(0, 5.0, "lda_md", LDA(1.0)))
lg = [acc(0, 0.5, "lda_md", LDA(g)) for g in hw.GAMMAS]
assert all(15 < x - lg[0] < 25 for x in lg[1:]), "every gamma > 0 gains about 20 points"
def four(s):
    """Test accuracy at q = 100 of the four MD pipelines (50/50, original features)."""
    return {"md": acc(0, s, "raw_md", MD100), "pca": acc(0, s, "pca_md", PCA2),
            "lda": acc(0, s, "lda_md", LDA(0.0)), "rlda": acc(0, s, "lda_md", LDA(1.0))}


for s in hw.IRIS_VARS:
    a = four(s)
    assert a["lda"] < a["rlda"], f"50/50: rLDA beats LDA at sigma2 = {s}"
    if s <= 0.5:
        assert max(a, key=a.get) == "pca", f"PCA2 is best at low noise (sigma2 = {s})"
    if s >= 2:
        assert max(a, key=a.get) == "rlda", f"rLDA is best at sigma2 = {s}"
lo, hi = four(0.01), four(0.5)
assert max(lo, key=lambda k: lo[k] - hi[k]) == "md", "MD on 104 dimensions falls fastest"
v["ilda20_005"], v["irlda20_005"] = (f1(acc(0, 0.05, "lda_md", LDA(0.0), T20)),
                                     f1(acc(0, 0.05, "lda_md", LDA(1.0), T20)))
assert float(v["ilda20_005"]) > float(v["irlda20_005"]), "20/80 at low noise: LDA edges out rLDA"
```

```
rows, table = [], (("MD，104 維", "raw_md", MD100), ("PCA2 + MD", "pca_md", PCA2),
                   ("LDA + MD", "lda_md", LDA(0.0)), (r"rLDA $\gamma=0.1$ + MD", "lda_md", LDA(0.1)),
                   (r"rLDA $\gamma=1$ + MD", "lda_md", LDA(1.0)), (r"rLDA $\gamma=10$ + MD", "lda_md", LDA(10.0)))
for name, key, pick in table:
    rows.append(f"{name} & " + " & ".join(f1(acc(0, s, key, pick, f)) for f in (T50, T20) for s in (0.05, 0.5, 5.0))
                + r" \\")
v["table2"] = "\n".join(rows)
lower = sum(acc(0, s, key, pick, T20) < acc(0, s, key, pick) for _, key, pick in table for s in (0.05, 0.5, 5.0))
assert lower > 3 * len(table) / 2, "most 20/80 entries of table 2 are lower than 50/50"
v["ipcaz_001"], v["ipca1_001"], v["ipcaz1_001"] = (f1(acc(1, 0.01, "pca_md", PCA2)), f1(acc(0, 0.01, "pca_md", PCA1)),
                                                   f1(acc(1, 0.01, "pca_md", PCA1)))
assert abs(acc(1, 0.01, "pca_md", PCA2) - acc(1, 0.01, "pca_md", PCA1)) < 0.5
assert all(acc(1, s, "pca_md", PCA2) < acc(0, s, "pca_md", PCA2) for s in hw.IRIS_VARS if s <= 0.2)
lamz, vecz = hw.eig_desc(hw.cov((iris - iris.mean(0)) / iris.std(0)))
v["zload"] = "、".join(f"${w:.2f}$" for w in vecz[:, 0] * np.sign(vecz[:, 0].sum()))
v["varpl"] = f"{iris[:, 2].var():.2f}"
P = np.bincount(target) / len(target)
for j, name in ((1, "fswidth"), (2, "fplength")):
    col = iris[:, j]
    within = sum(P[c] * col[target == c].var() for c in range(3))
    between = sum(P[c] * (col[target == c].mean() - col.mean()) ** 2 for c in range(3))
    v[name] = f"{between / within:.2f}" if between < within else f"{between / within:.1f}"

# Run details for the appendix
v["seconds"] = f"{INFO['seconds']:.0f}"
for key in ("python", "numpy", "scipy", "scikit_learn"):
    v[key] = INFO[key]
import matplotlib
v["matplotlib"] = matplotlib.__version__


def fill(template, extra):
    values = {**v, **extra}
    missing = set(re.findall(r"<<(\w+)>>", template)) - values.keys()
    assert not missing, f"no value for {sorted(missing)}"
    return re.sub(r"<<(\w+)>>", lambda m: values[m.group(1)], template)


def build(template, extra, dest):
    (REPORT / "report.tex").write_text(fill(template, extra))
    for _ in range(2):
        subprocess.run(["xelatex", "-interaction=nonstopmode", "-halt-on-error", "report.tex"],
                       cwd=REPORT, check=True, stdout=subprocess.DEVNULL)
    aux = (REPORT / "report.aux").read_text()
    body = int(re.search(r"\\newlabel\{body-end\}\{\{[^}]*\}\{(\d+)\}", aux).group(1))
    assert body <= 5, "the report body must fit in five pages"
    shutil.copyfile(REPORT / "report.pdf", dest)
    print(f"{dest.name}: body ends on page {body}")


def leaks(pdf, secret):
    """True if secret shows up in the PDF text, its metadata, or its raw bytes (ASCII or UTF-16BE)."""
    text = "".join(subprocess.run([tool, str(pdf), *extra], capture_output=True, text=True).stdout
                   for tool, extra in (("pdftotext", ["-"]), ("pdfinfo", [])))
    raw = pdf.read_bytes()
    utf16 = secret.encode("utf-16-be")
    return secret in text or secret.encode() in raw or utf16 in raw or utf16.hex().upper().encode() in raw.upper()


def main():
    template = (REPORT / "template.tex").read_text()
    unused = set(v) - set(re.findall(r"<<(\w+)>>", template))
    assert not unused, f"computed but never quoted: {sorted(unused)}"
    sid = json.loads(PRIVATE.read_text())["student_id"] if PRIVATE.exists() else None
    if sid:  # submission copy for e3, kept local
        build(template, {"sid": "\u3000" + sid, "pdfsid": " " + sid}, HERE.parent / "submission" / f"PR_HW1_蔡秀吉_{sid}.pdf")
    build(template, {"sid": "", "pdfsid": ""}, PUBLIC_PDF)  # built last, so report/ ends without the ID
    if sid:
        assert not leaks(PUBLIC_PDF, sid), "the public PDF must not contain the student ID"
        assert all(sid not in f.read_text(errors="ignore") for f in REPORT.iterdir() if f.suffix != ".pdf")
```

```
        print(f"{PUBLIC_PDF.name}: no student ID in text, metadata or bytes")


if __name__ == "__main__":
    main()
```